app-agents — Install & Getting Started

Author real Kissflow apps from a BRD with a team of Claude agents — data models, workflows, roles, permissions, pages — reviewed before anything is built.

V1.0.0 July 2026 · works in Claude Code desktop, CLI and Cowork

1 What you need

- **Claude Code** (desktop app, CLI, or a Cowork session).
- A **Google account on the kissflow.com domain** — sign-in to the appbuilder is Google SSO, domain-restricted.
- A **Kissflow DEV account access key** (one person sets it up once for the team — see §3).
- Node 18+ (the build engine is zero-dependency; python3 only for native page rendering).

2 Install the plugin

```
/plugin marketplace add https://github.com/kissflow/ai-plugin.git
/plugin install app-agents@kissflow
```

> **Always install from the git URL, never a local folder.** Cowork/cloud sandboxes can't read your machine — a directory-sourced marketplace silently pins them to a stale copy of the plugin.

Updating later: `/plugin marketplace update ai-plugin` → `/plugin update app-agents` → restart Claude Code.

3 One-time team setup — the appbuilder

The team console lives at **appbuilder.zingworks.com**. Sign in with Google, then add a **dev environment** once (Dashboard → *Dev environments* → *Add*):

- **Subdomain**: your Kissflow dev/builder account, e.g. `dev-acme` (read it off the URL).
- **Account id**: Kissflow → Account settings → Account details.
- **Access key id + secret** (both required): Kissflow dev account → Account Administration → Integrations → Access Keys → Create. The secret is shown once — paste both immediately.

Credentials are stored server-side in Secret Manager and **never displayed again** (rows show a masked `key ••1234` hint). Every project can reuse the same environment; owners can rotate the key pair or delete the environment from the same list.

> **Use the DEV environment.** Dev/builder and runtime accounts have different ids and keys — a key from one gets `403 DomainMissMatchError` on the other. The pipeline never auto-publishes to prod.

4 Per-workspace setup — `/author-setup`

In the folder (or Cowork session) you'll author from, run `/author-setup` — **no arguments**. It stages the engine + playbooks + memory, then runs a browser handshake:

1. A verify link opens → sign in with Google.
2. Pick an existing **project** or create one on the spot, and pick its **dev environment**.
3. Approve → "Session connected" → *Continue to project*. The session is now fully wired: scoped dev credentials, your identity (versions and memory are attributed to you), the project's shared-memory partition, artifact storage — and `MEMORY-REMOTE.md` syncs the newest team knowledge from the server.

A pre-minted connect URL from the project page also works: `/author-setup <connect-url>` (single-use, expires in ~10 minutes — just mint a fresh one if it lapses).

5 Build an app

Express (demos, small/clear specs) — one command, no review pause:

```
/author-app "an employee onboarding app
  with HR approval and IT provisioning"
```

Runs brief → plan → generate back-to-back and builds the real app in dev, typically in a few minutes. `--dry-run` / `--yes` supported.

Staged (recommended for real BRDs) — review before anything is built:

```
/author-brief <brd>   ingest → new run
/author-plan          propose design (v1)
/author-review        review + prototype
/author-refine "…"    iterate (vN+1)
/author-preview       dry-run gate
/author-generate      build in dev
```

`/author-review` renders an interactive review page (every field, workflow step, permission and decision is flaggable) plus a **clickable per-role prototype** — both are registered on the appbuilder project page for the team to open and judge. `/add-flow` / `/add-board` extend an existing app through the same pipeline.

6 Where the work shows up

On the project page	What it tells you
Versions	Every build and review is snapshotted — the review page and prototype open directly from here.
✓ connected badge	Who last attached a Cowork session, and when.
Members	Owner / builder / viewer roles; builders can mint connect links.
Danger zone	Owners can delete the project (type-name confirm). The built Kissflow app and dev environments survive.

7 The plugin keeps learning

Agents read three memory layers before working: the canonical `MEMORY.md` (ships with the plugin), `MEMORY-LOCAL.md` (your workspace), and `MEMORY-REMOTE.md` — synced on every connect from the team's shared pool, so every session starts with the newest platform knowledge. Lessons are written back with `node engine/memory.mjs remember "…"`. Claims that something is *impossible* are quarantined until the platform owner confirms them — one wrong belief can't mislead the whole team.

8 Troubleshooting

Symptom	Fix
Connect link says expired / already used	They're single-use, ~10 min. Re-run `/author-setup` or mint a fresh link on the project page.
Env row shows ⚠ **secret missing**	The stored key has no secret — Edit (✎) and paste the full key pair.
`403 DomainMissMatchError` on builds	Key and subdomain are from different environments — use the DEV account for both (§3).
Plugin shows an old version	`/plugin marketplace update ai-plugin` → `/plugin update app-agents` → restart. In Cowork, start a fresh session.
Dev-env delete refused	Projects still link it — the error lists them; relink those projects first.

app-agents v1.0.0 · install: **github.com/kissflow/ai-plugin** (kissflow plugins marketplace) · console: **appbuilder.zingworks.com** · full agent-use guide: `docs/PIPELINE.md` · authored by Dinesh V — companion to kf-app-builder (the React-UI pipeline).